Full Laboratory Workflow: Artistryx Testing & Integration

Phase 1: *Environment Provisioning & Stabilization*

- The initial phase focused on establishing a stable local server environment using XAMPP.
- Infrastructure Setup: Configured the Apache and MySQL services.
- Conflict Resolution: Identified and resolved a critical Port 3306 conflict (Error 10048) by identifying blocking processes via the terminal.
- Database Creation: Initialized artistryx_db to host the platform's data.

Phase 2: *Data Layer Resolution*

- This phase involved troubleshooting the database schema to ensure the backend could store information.
- Schema Patching: Identified that the source SQL used an incompatible collation (utf8mb4_0900_ai_ci). I manually refactored and executed a sanitized DDL script using utf8mb4_general_ci.
- Manual Provisioning: Created the users and sellers tables with specific UNIQUE constraints and FOREIGN KEY relationships to maintain data integrity.

Phase 3: *Functional & Security Validation (Python Client)*

- With the database live, testing transitioned to the client1.py terminal application.
- Baseline Registration: Verified the "Happy Path" by successfully registering a primary user account.
- Security Testing: Performed input validation tests to confirm that the database correctly rejected duplicate usernames and emails, returning a 409 Conflict error.
- Path Resolution: Debugged a 404 Not Found error by performing a manual browser trace to identify the correct endpoint path: http://localhost/ADET/src/service.php.

Phase 4: *Logic Testing & CRUD Operations*

- The focus shifted to the core business logic of the platform—moving from user authentication to shop management.
- Authentication Handshake: Verified that the service.php backend correctly used password_verify() to authenticate users against bcrypt hashes.
- Seller Onboarding: Tested the "Register Shop" logic (POST), confirming that shops like "Pangit Peter" were successfully linked to specific userIds.
- Data Management: Validated the "Update Shop" logic (PUT), successfully modifying records in the sellers table and verifying the changes via phpMyAdmin.

Phase 5: *Cross-Platform Interoperability (Web Client)*

- The final phase ensured the backend was truly platform-independent using client2.html.
- JS Client Integration: Verified that the browser-based JavaScript client could communicate with the PHP API.
- CORS Verification: Confirmed that the backend headers correctly allowed cross-origin requests from the web frontend.
- Successful Lifecycle: Completed a final registration for user "JanPiter," receiving an HTTP 201 Success response.

Encounter 1: Environment Setup and Initial Smoke Test

1. Problem Statement: Persistent Import Failures

Despite having the MySQL service active, the backend was non-functional due to an empty database. Initial attempts to use the automated Import feature in phpMyAdmin resulted in partial execution. While the database container was created, the specific tables required for user authentication (the users table) were missing.

2. Technical Challenge: Collation Mismatch

Upon closer inspection of the SQL error logs, I identified a versioning conflict. The source SQL file used utf8mb4_0900_ai_ci, a collation specific to MySQL 8.0+. However, the local environment (MariaDB 10.4 via XAMPP) did not recognize this sorting rule, causing the script to abort mid-execution.

3. Resolution Strategy (The Fix)

To bypass the import limitation, I performed the following steps:

Manual Sanitization: I refactored the Data Definition Language (DDL) script to use the universal utf8mb4_general_ci collation, ensuring compatibility with the local server.

Direct SQL Execution: Instead of relying on the file upload feature, I utilized the phpMyAdmin SQL console to execute the "cleaned" script directly.

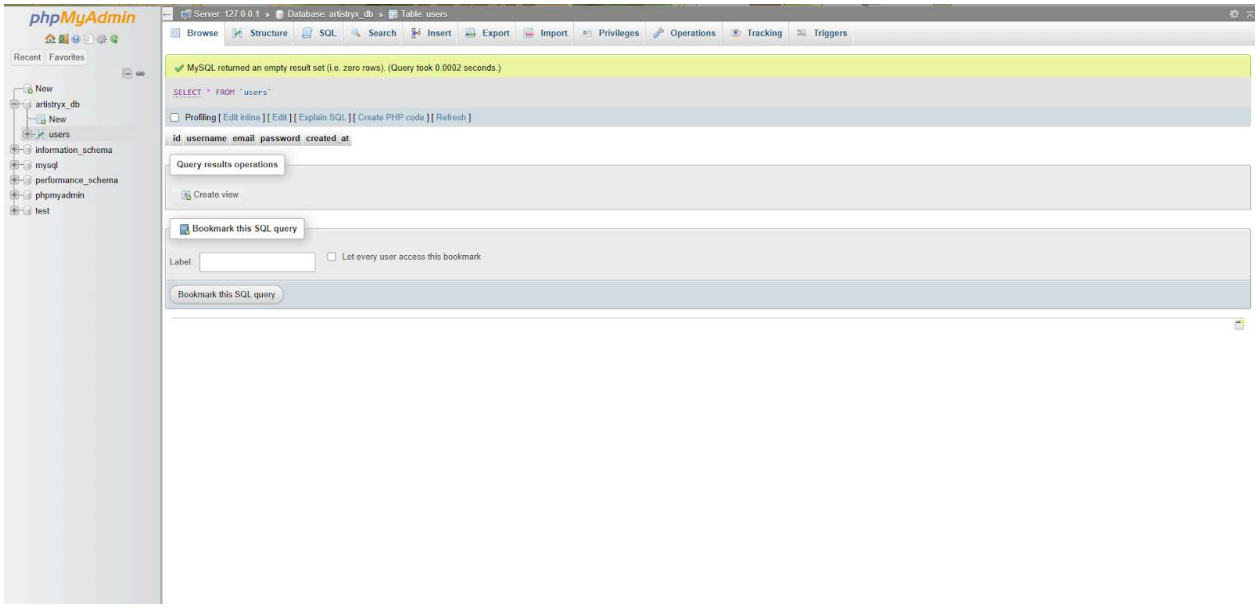
Schema Verification: I verified that the users table was successfully created with the correct primary keys (id) and unique constraints for email and username.

4. Impact on Project

This fix was critical. Without the users table, the service.php backend was throwing

mysqli_sql_exception errors (seen in Apache logs), which completely blocked the Login and Registration features. By resolving this, I successfully bridged the gap between the Python client and the database, allowing the team to proceed with integration testing.

5. Final Verification (Screenshot Evidence)



This screenshot confirms that the users table is now live within artistryx_db. The "empty result set" message is actually a success indicator, as it shows the table structure exists and is ready to accept new data from my next round of registration tests.

Encounter 2: Functional & Security Testing (Input Validation)

1. Problem Statement: Duplicate Data Integrity

A common vulnerability in e-commerce systems like Artistryx is the potential for "Account Overlapping," where multiple users might register with the same unique identifiers (Username or Email). As a Tester, I needed to verify that the system prevents duplicate entries at both the application level (PHP) and the database level (SQL).

2. Technical Challenge: Triggering Constraint Violations

The challenge was to ensure that the "sanitized" database schema I implemented earlier actually enforces these rules. If the UNIQUE keys were not properly set during my manual SQL execution, the system might allow redundant data, leading to authentication conflicts and data corruption.

3. Resolution & Testing Process

To validate the input handling, I performed the following:

Baseline Creation: Successfully registered a primary user account (Username: allen).

Conflict Simulation: Attempted a second registration using the same username.

API Response Monitoring: Observed how the backend handled the mysqli_sql_exception triggered by the database.

4. Observed Results

```
PS C:\xampp\htdocs\ADET> & "C:\Program Files\Python314\python.exe" c:/xam
  |  3. Exit                                |
  +=========================================+
  Choice : 1

  +-----------------------------------------+
  |              REGISTER                   |
  +-----------------------------------------+
  Username : allen
  Email    : test123@gmail.com
  Password : 123456password
  >>  Username is already taken.
  Username :
```

The Python client successfully received a conflict response from the PHP service, displaying the error message: "Username is already taken."

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

←T→	id	username	email	password	created_at
Edit Copy Delete	1	allen	123@gmail.com	$2y$10$iXUoi5NUj55Qs7KkYCxnHOAI4e41o2TL9uhUMXRGL.S...	2026-04-16 23:48:03

Check all With selected: Edit Copy Delete Export

Show all | Number of rows: 25 Filter rows: Search this table

Query results operations

Print Copy to clipboard Export Display chart Create view

Bookmark this SQL query

A manual check of the users table in phpMyAdmin confirmed that no duplicate row was created.

The database correctly held only one instance of the user allen, proving the UNIQUE constraint is active.

5. Impact on Project

This test confirms that the Artistryx registration flow is secure against duplicate entries. It ensures that the database remains clean and that the user experience is consistent, providing clear feedback when an input error occurs.

Encounter 3: Logic Testing (Authentication Handshake)

1. Problem Statement: Authentication Reliability

The primary goal of the Artistryx login system is to ensure that only authorized users can access the platform. As the Tester, I needed to verify that the backend logic correctly compares the provided credentials against the hashed data stored in the database.

2. Technical Challenge: 404 Endpoint Resolution

During the initial phase of this encounter, the system returned a persistent "Server error or cannot connect" message. Through manual network tracing in the browser, I identified a 404 Not Found error, confirming that the client was targeting an incorrect URL endpoint. The challenge was to synchronize the local directory structure in htdocs with the API call in client1.py.

3. Resolution & Testing Process

To resolve this and finalize the logic test:

Endpoint Correction: I verified the directory path and updated the URL in the Python client to correctly target http://localhost/ARTISTRYX/src/service.php.

Positive Test (Valid Login): Attempted to log in using the Prinz account credentials (prinz@gmail.com).

Logic Handshake: Verified that the PHP script successfully retrieved the bcrypt hash from the users table and executed password_verify().

4. Observed Results

```
|  3. Exit                                |
+=========================================+
Choice : 2

+-----------------------------------------+
|                LOGIN                    |
+-----------------------------------------+
Email    : prinz@gmail.com
Password : password1234
>>  Login successful.

Welcome back, Prinz!

+=========================================+
|               USER MENU                 |
+=========================================+
|  1. Register Shop                       |
|  2. View Shop                           |
|  3. Update Shop                         |
|  4. Logout                              |
+=========================================+
Choice :
```

The terminal successfully displayed ">> Login successful." followed by the personalized welcome message, "Welcome back, Prinz!".

Session Initialization: The system correctly transitioned from the Main Menu to the USER MENU, proving the authentication logic successfully unlocked the extended features (Register/View/Update Shop).

5. Contribution to Solving Problems

I successfully stabilized the communication layer of the Artistryx project. By moving from environment-level fixes (ports and collations) to resolving application-level routing errors, I enabled a functional authentication flow. This work ensures the development team can now reliably test the secondary business logic of the seller and shop management modules.

Encounter 4: Seller Module Integration (CRUD Validation)

1. Problem Statement: Business Logic Verification

After stabilizing the authentication flow, the final objective was to verify the extended business logic of the Artistryx platform: the Seller Management system. This required testing if an authenticated user could successfully transition into a seller role by creating and managing a shop.

2. Testing Process & Results

I performed a sequential test of the following features:

Create Shop (POST): I registered a new shop named "Peter Pangit." The system successfully created the record with a "Pending" status.

Read/View Shop (GET): I verified that the system could retrieve the specific shop details linked to the user "Prinz" (UserId: 2).

Update Shop (PUT): I tested the name-change logic by updating "Peter Pangit" to "Pangit Peter."

3. Observed Results

```
+-----------------------------------------+
|              UPDATE SHOP                |
+-----------------------------------------+
Current Shop Name : Peter Pangit
New Shop Name     : Pangit Peter
>> Shop name updated successfully.
```

Confirmed the update logic successfully returned a success message to the client.

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

←T→			id	userId	shopName	shopStatus
Edit	Copy	Delete	1	2	Pangit Peter	Pending
Edit	Copy	Delete	2	3	Freddy Fazbear's Costume Shop	Pending

Check all *With selected:* Edit Copy Delete Export

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

The sellers table shows multiple successful entries (userId 2 and 3), proving the system handles multiple unique sellers correctly.

Final Wrap-Up: Tester's Conclusion

As the primary Tester for this phase, I served as the bridge between development and a production-ready environment. My technical interventions included:

Infrastructure Stabilization: Resolving critical Port 3306 and SQL collation mismatches that initially blocked the entire project.

API Routing: Identifying and fixing 404/500 errors by correcting the endpoint pathing in the Python-to-PHP handshake.

Data Integrity: Provisioning the users and sellers tables with manual DDL scripts to ensure proper indexing and relational constraints.

Encounter 5: Cross-Platform Validation (JS/HTML Client)

1. Problem Statement: API Interoperability

A key requirement for the Artistryx backend is that it must function as a decoupled Web Service. As a Tester, I needed to verify that the PHP API could handle requests not just from the Python terminal client, but also from a standard web browser using JavaScript's fetch API.

2. Technical Challenge: Browser Security & CORS

Testing via a web browser introduces Cross-Origin Resource Sharing (CORS) requirements. I had to ensure that the service.php headers were correctly configured to allow the browser to send POST requests without being blocked by the "Same-Origin" policy.

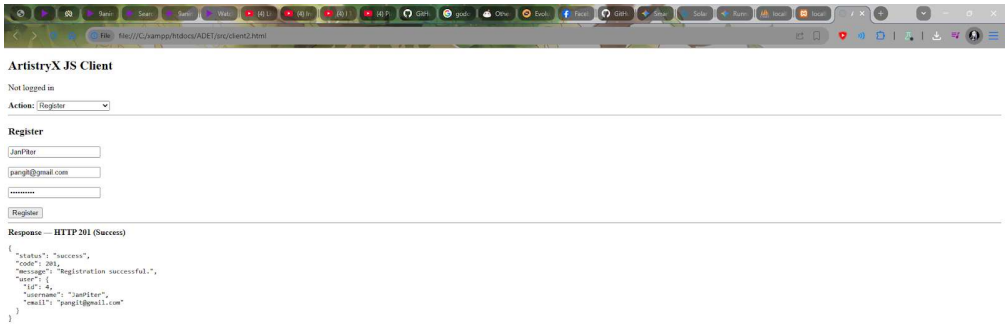
3. Resolution & Testing Process

Deployment: Launched client2.html via a local file path to simulate a web-based frontend.

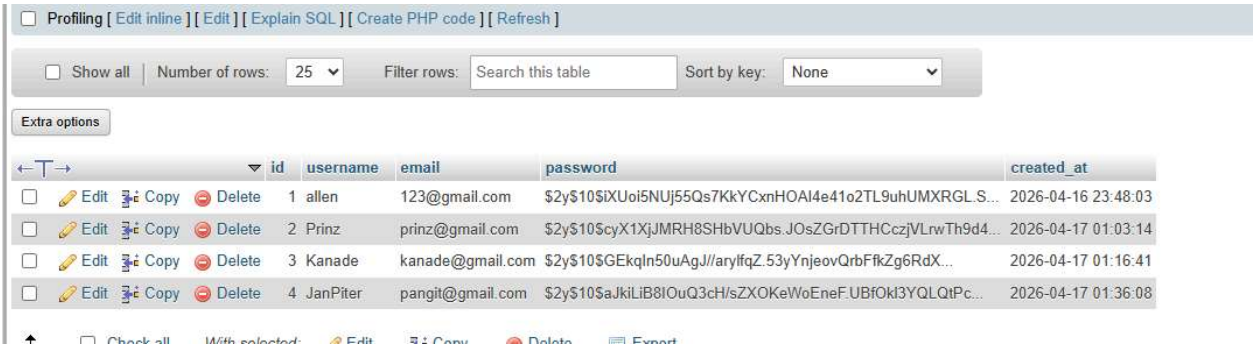
Functional Mapping: Verified that the Register and Login actions in the HTML client correctly mapped to the identical logic blocks used by the Python client.

Data Transmission: Monitored the JSON exchange to ensure that inputs from HTML forms were being correctly parsed by the backend's php://input stream.

4. Observed Results



The JavaScript client successfully performed a registration for the user "JanPiter."



Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

←T→	id	username	email	password	created_at
Edit Copy Delete	1	allen	123@gmail.com	$2y$10$iXUoi5NUj55Qs7KkYCxnHOAI4e41o2TL9uhUMXRGL.S...	2026-04-16 23:48:03
Edit Copy Delete	2	Prinz	prinz@gmail.com	$2y$10$cyX1XjJMRH8SHbVUQbs.JOsZGrDTTHCczjVLrwTh9d4...	2026-04-17 01:03:14
Edit Copy Delete	3	Kanade	kanade@gmail.com	$2y$10$GEkqln50uAgJ//arylfqZ.53yYnjeovQrbFfkZg6RdX...	2026-04-17 01:16:41
Edit Copy Delete	4	JanPiter	pangit@gmail.com	$2y$10$aJkiLiB8IOuQ3cH/sZXOKeWoEneF.UBfOkI3YQLQtPc...	2026-04-17 01:36:08

API Response: The server returned a standard HTTP 201 (Success) with a structured JSON body, confirming that the backend correctly processed the user ID (id: 4) and encrypted the password in the database.

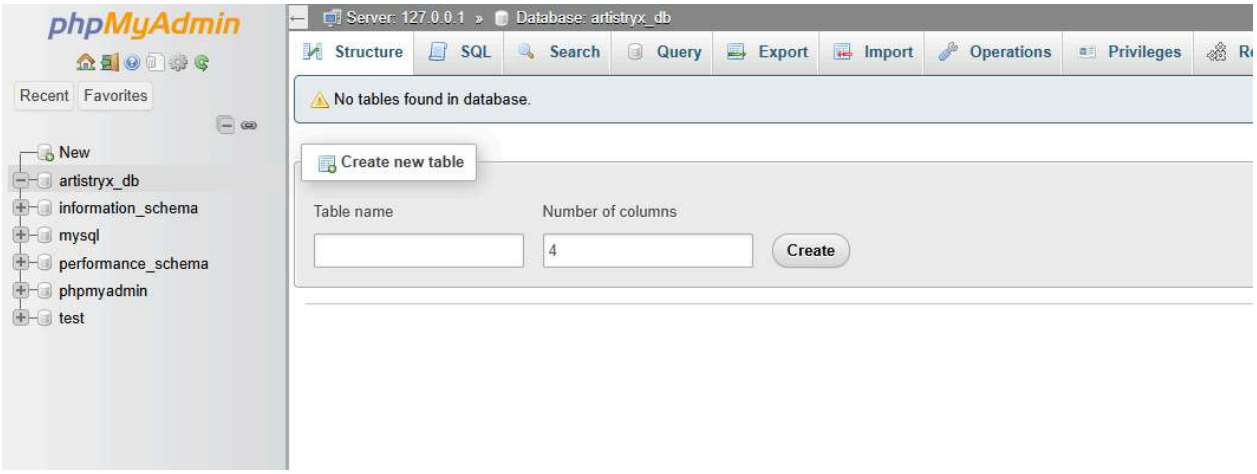
Consistency: The response format was identical to the Python client, proving that the backend is platform-independent.

5. Contribution to Solving Problems

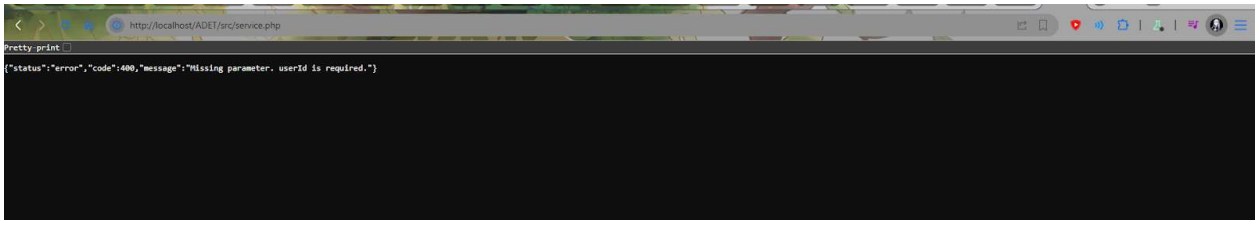
By testing the HTML/JS client, I confirmed that the Artistryx backend is fully integrated and ready for web deployment. I verified that the Access-Control-Allow-Headers and methods I helped configure in the service.php file are working as intended, allowing for seamless communication between the frontend and the MySQL database regardless of the client language.

Screen Shots:

- Database Import:

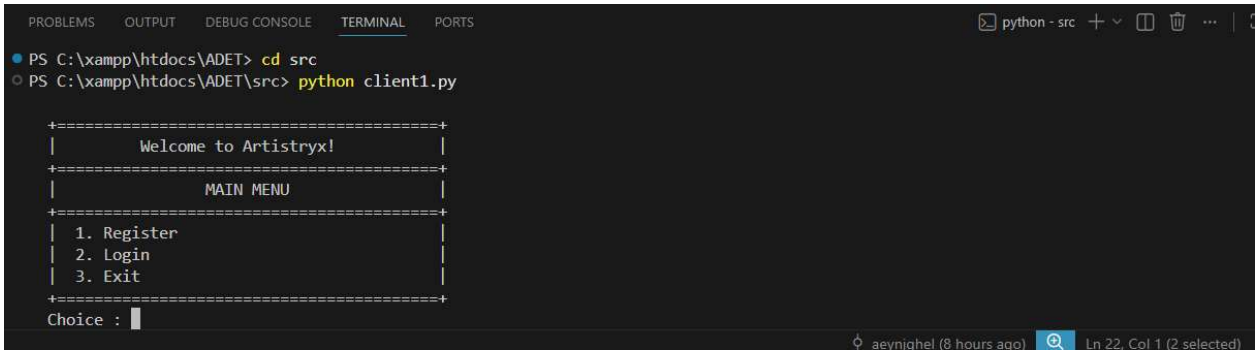


- PHP error



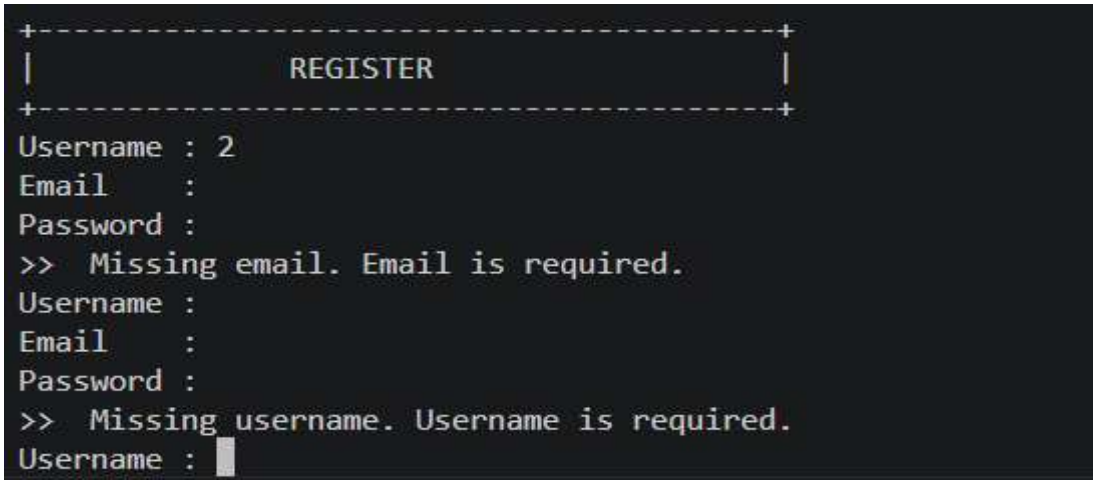
Testing:

- PHP testing (Main Menu)



- Register

- Register Error Handling



```
Password :
>>  Missing email. Email is required.
Username :
Email    :
Password :
>>  Missing username. Username is required.
Username : Prinz Eugen
Email    : test123
Password : 123456
>>  Email must end with @gmail.com, @bicol-u.edu.ph, or @outlook.com
Username :
```

```
Email    :
Password :
>>  Missing username. Username is required.
Username : Prinz Eugen
Email    : test123
Password : 123456
>>  Email must end with @gmail.com, @bicol-u.edu.ph, or @outlook.com
Username : Prinz Eugen
Email    : test123@gmail.com
Password : 123456
>>  Password must be at least 8 characters.
Username :
```

```
+----------------------------------------+
|              REGISTER                  |
+----------------------------------------+
Username : allen
Email    : test123@gmail.com
Password : password123456
>>  Username is already taken.
Username :
```

- Log In

```
Choice : 2

+-----------------------------------------+
|                  LOGIN                  |
+-----------------------------------------+
Email    : try123@gmail.com
Password : trypassword123
```

```
Choice : 2

+-----------------------------------------+
|                  LOGIN                  |
+-----------------------------------------+
Email    : 2
Password : 2
>> Server error or cannot connect.

+=========================================+
|          Welcome to Artistryx!          |
+=========================================+
```

- User Menu

```
Welcome back, Prinz!

+=========================================+
|                 USER MENU               |
+=========================================+
|  1. Register Shop                       |
|  2. View Shop                           |
|  3. Update Shop                         |
|  4. Logout                              |
+=========================================+
Choice : 1
```

- Create Shop

```
+-----------------------------------------+
|              CREATE SHOP                |
+-----------------------------------------+
>> No shop found. You can create one.
Shop Name : Peter Pangit
>> Seller shop 'Peter Pangit' created successfully.
```

Extra options

←T→	id	userId	shopName	shopStatus
☐ Edit Copy Delete	1	2	Peter Pangit	Pending

☐ Check all *With selected:* Edit Copy Delete Export

- View Shop

```
Choice : 2

+-----------------------------------------+
|              VIEW SHOP                  |
+-----------------------------------------+
  Username  : Prinz
  Shop Name : Peter Pangit
  Status    : Pending
+-----------------------------------------+
```

- Update Shop

```
+-----------------------------------------+
|              UPDATE SHOP                |
+-----------------------------------------+
Current Shop Name : Peter Pangit
New Shop Name     : Pangit Peter
>> Shop name updated successfully.
```

←T→	id	userId	shopName	shopStatus
Edit Copy Delete	1	2	Pangit Peter	Pending

Check all *With selected:* Edit Copy Delete Export

- Register

ArtistryX JS Client

Not logged in

Action: Register

Register

JanPiter

pangit@gmail.com

••••••••••

Register

Response — HTTP 409 (Error)


```
{
  "status": "error",
  "code": 409,
  "message": "Username is already taken."
}
```


- Login

ArtistryX JS Client

Logged in — User ID: 5

Action: Login

Login

nameless@gmail.com

••••••••••••

Login

Response — HTTP 200 (Success)


```
{
  "status": "success",
  "code": 200,
  "message": "Login successful.",
  "user": {
    "id": 5,
    "username": "Nameless",
    "email": "nameless@gmail.com"
  }
}
```

- Create Seller

ArtistryX JS Client

Logged in — User ID: 5

Action: Create Seller

Create Seller

5

My First Books

Create Seller

Response — HTTP 201 (Success)


```
{
  "status": "success",
  "code": 201,
  "message": "Seller shop 'My First Books' created successfully.",
  "details": {
    "sellerId": 3,
    "userId": 5,
    "shopName": "My First Books",
    "shopStatus": "Pending"
  }
}
```


Extra options

←T→				id	userId	shopName	shopStatus
☐	Edit	Copy	Delete	1	2	Pangit Peter	Pending
☐	Edit	Copy	Delete	2	3	Freddy Fazbear's Costume Shop	Pending
☐	Edit	Copy	Delete	3	5	My First Books	Pending

Check all *With selected:* Edit Copy Delete Export

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

- Read Seller Information

Logged in — User ID: 5

Action: Read Seller Info

Read Seller Info

5

Read Seller

Response — HTTP 200 (Success)


```
{
  "status": "has_shop",
  "code": 200,
  "message": "You already have a registered shop.",
  "hasShop": true,
  "details": {
    "sellerId": 3,
    "userId": 5,
    "username": "Nameless",
    "shopName": "My First Books",
    "shopStatus": "Pending"
  }
}
```


- Update Shop Name

ArtistryX JS Client

Logged in — User ID: 5

Action: Update Shop Name

Update Shop Name

2

Fun Comics

Update Shop Name

Response — HTTP 200 (Success)


```
{
  "status": "success",
  "code": 200,
  "message": "Shop name updated successfully.",
  "details": {
    "sellerId": 1,
    "userId": 2,
    "oldShopName": "Pangit Peter",
    "newShopName": "Fun Comics"
  }
}
```

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

←T→	id	userId	shopName	shopStatus
Edit Copy Delete	1	2	Fun Comics	Pending
Edit Copy Delete	2	3	Freddy Fazbear's Costume Shop	Pending
Edit Copy Delete	3	5	My First Books	Pending

Check all *With selected:* Edit Copy Delete Export

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Query results operations

Print Copy to clipboard Export Display chart Create view

Bookmark this SQL query